

Parallelization of Barnes–Hut N-Body Simulation Using OpenMP

Arnav Gupta
ag11163@nyu.edu

April 16, 2026

Abstract

The gravitational N-body problem is fundamental to astrophysical simulations. The naive direct summation approach has $O(N^2)$ complexity and becomes extremely expensive for large particle counts (greater than the order of millions). The Barnes-Hut algorithm reduces complexity to $O(N \log N)$ via hierarchical tree-based approximation. However, parallelizing Barnes-Hut is challenging due to its irregular tree structure and data traversal patterns, both due to imbalances arising from clustered particle distributions.

We implement a complete 2D Barnes-Hut N-body simulator parallelize it using OpenMP with two key innovations: (1) Morton key spatial sorting of particles to improve cache locality, and (2) task-based parallel tree construction with lock-free synchronization.

Our experiments with up to 1M particles demonstrate that force computation exceeds naive Amdahl predictions due to cache locality improvements from Morton sorting. The approximation parameter θ controls total sequential work but does not affect parallel scaling efficiency, indicating our parallelization techniques generalize across accuracy regimes.

1 Introduction

1.1 Motivation and Importance of the N-Body Problem

The gravitational N-body problem is one of the oldest and most fundamental challenges in computational science. The direct summation approach, where each particle’s force is computed by summing contributions from all other particles, is exact but has $O(N^2)$ complexity. For $N = 10^6$ particles, this requires 10^{12} force evaluations per timestep. The Barnes-Hut algorithm [1] dramatically improves scalability by exploiting spatial locality: distant particles are grouped hierarchically and approximated as a single point mass, reducing complexity to $O(N \log N)$.

However, scaling Barnes-Hut to multiple cores is non-trivial. The irregular tree structure and data-dependent access patterns make load balancing and cache optimization challenging. Each particle’s traversal path through the tree is determined at runtime based on its position and the opening angle criterion. The CPU cannot predict which tree nodes will be accessed, making hardware prefetching ineffective. This leads to long memory latency stalls. This project investigates how to effectively parallelize Barnes-Hut on shared-memory systems (OpenMP) and identifies the key factors that drive parallel performance. Our approach sorts particles by their Morton [2] key (Z-order curve) before tree construction, enabling us to be able to divide tree construction amongst threads. We subdivide the trees amongst threads for the top levels only, then switch to sequential insertion which helps us avoid task overhead for smaller subtrees

2 Literature Survey

2.1 Approximation Methods

The fundamental N-body problem can be approached in two ways, multipole expansion and hierarchical tree methods.

2.1.1 Fast Multipole Method (FMM)

Greengard and Rokhlin [3] introduced FMM, which achieves $O(N)$ complexity by expressing the potential as a multipole expansion. It is more accurate than Barnes-Hut for fixed computational cost but is significantly more complex to implement and has higher constant factors, making it the method of choice for extremely large systems (billions of particles), but overkill for moderately large systems ($N \lesssim 10^7$) where Barnes-Hut suffices.

2.1.2 Barnes-Hut Algorithm

Barnes and Hut [1] introduced the tree-based $O(N \log N)$ algorithm that is the subject of this project. The algorithm partitions space hierarchically into quadrants (2D) or octants (3D), inserting particles one-by-one and computing center-of-mass for each node. During force computation, an opening angle criterion θ determines whether a node is treated as a single point or recursively traversed. Smaller values of θ result in simulations that are more accurate but slower.

2.2 Category 2: Shared-Memory Parallelization (OpenMP)

Shared-memory approaches assume all threads access a single unified address space without explicit message passing. Most student projects and textbook implementations parallelize only force computation using `parallel for` with static or dynamic scheduling. Tree construction remains sequential. It provides good speedup for force computation (typically 10-20 $\times$ on 64 cores), low synchronization overhead.

2.3 Category 3: Distributed-Memory Parallelization (MPI)

Warren and Salmon [4] introduced the hashed oct-tree: particles are sorted by Morton key (Z-order curve), partitioned across nodes, and tree construction at each node can proceed independently (within a communication phase). The key insight of spatial ordering enabling natural decomposition is central to our shared-memory approach.

2.4 Gap Analysis: Why Our Work is Needed

The literature spans many areas, but a specific gap remains:

1. **Limited shared-memory analysis:** The literature we reviewed focuses on distributed-memory (MPI). Shared-memory OpenMP implementations in academic literature are sparse; most are textbook exercises without detailed performance analysis.
2. **Data layout and cache effects underexplored:** Warren & Salmon [4] introduced Morton keys for load balancing in MPI codes. However, the cache locality benefits in shared-memory settings have not been quantified in detail. We measure LLC miss rates, IPC, and effective serial fraction—metrics absent from prior shared-memory work.
3. **Decoupling of work and scaling:** No prior work explicitly shows that the approximation parameter θ controls total work but not parallel scaling efficiency. This is important for practitioners: it means scheduling strategies are robust across accuracy regimes.

4. **Amdahl’s law superlinearity:** Measured speedups exceeding Amdahl predictions are documented in some contexts (e.g., cache effects in matrix multiplication on small caches), but are rare in discussion of tree-based algorithms. Our analysis explains when and why superlinear speedups occur.

Our project addresses these gaps by providing a detailed empirical and analytical study of shared-memory Barnes-Hut parallelization, with emphasis on cache optimization and hardware metrics.

3 Proposed Idea

Our implementation consists of the following components

- The Barnes-Hut force approximation algorithm using quadtrees
- Morton key spatial sorting of particles
- The KDK leapfrog method used to numerically integrate the differential equations of motion
- Initializing the particles based on the Plummer model
- An OpenMP-based strategy to parallelize some of the components mentioned above

3.1 The Barnes-Hut Algorithm

The Barnes-Hut algorithm [1] is an approximation algorithm known to have an order $O(N \log N)$ as compared to traditional $O(N^2)$. It does this by hierarchically grouping distant particles and approximating their collective gravitational effect as a single point mass.

3.1.1 Quadtree Construction

We operate in two dimensions, using a quadtree to recursively subdivide space into four quadrants. Starting from a root cell that encompasses all particles, each cell is subdivided whenever it contains more than one particle. The process continues until every leaf cell contains at most one particle. Particles are inserted one at a time from the root downward. Three cases arise during insertion:

1. **Empty leaf:** The leaf stores the particle
2. **Occupied leaf:** The leaf is converted to an internal node, and both the existing and new particles are inserted into the appropriate quadrant.
3. **Internal node:** The particle is routed to the appropriate quadrant based on its position relative to the center of the internal node.

We might also encounter a degenerate case where the initializer loads two particles in exactly the same position. For this, we have hard-coded a maximum tree depth of 64 levels to prevent infinite recursion

3.1.2 Center-of-Mass Computation

After the tree is constructed, a bottom-up traversal (starting from the leaves, all the way up to the root through each internal node), computes the total mass M and center of mass $\mathbf{r}_{\text{com}}$ for each node:

$$M = \sum_i m_i, \quad \mathbf{r}_{\text{com}} = \frac{1}{M} \sum_i m_i \mathbf{r}_i \quad (1)$$

where the summation is over all the particles in the current subtree rooted at an internal node (or leaf). For leaf nodes, M and $\mathbf{r}_{\text{com}}$ are trivially the particle’s mass and position.

3.1.3 Opening Angle Criterion and subsequent Force Computations

For each particle i , we traverse the tree starting from the root. At each internal node, we apply the Barnes-Hut acceptance criterion to decide whether the node’s contents can be approximated as a single body:

$$\frac{s}{d} < \theta \quad (2)$$

where s is the side length of the cell and d is the distance from particle i to the cell’s center of mass, and θ is a user-defined accuracy parameter. If the criterion is satisfied, we compute the gravitational acceleration from the cell as:

$$\mathbf{F}_i += G M m_i \frac{\mathbf{r}_{\text{com}} - \mathbf{r}_i}{(|\mathbf{r}_{\text{com}} - \mathbf{r}_i|^2 + \epsilon^2)^{3/2}} \quad (3)$$

where ϵ is a gravitational softening parameter that prevents the force from blowing up when the separation is too small [7]. If the criterion is not satisfied, we recurse into the node’s four children and check recursively four different force contributions. Note that if we reach the leaf nodes, we are essentially computing direct pairwise interactions between particles.

We use $\theta = 0.5$ as the default, which provides a good balance between accuracy and speed but also vary it as part of our experiments to study the effect on the time taken to compute the total forces. We use $\epsilon = 0.025$ in reduced units.

3.2 Morton Key Sorting for Cache Optimization

Before each tree construction, we sort particles by their Morton key, following the approach introduced by Warren and Salmon [4]. The Morton key maps a 2D position (x, y) to a single 64-bit integer by interleaving the bits of the quantized coordinates:

$$\text{key} = \dots y_2 x_2 y_1 x_1 y_0 x_0 \quad (4)$$

Sorting by Morton key achieves two benefits:

1. **Cache-friendly force computation:** When iterating over particles in Morton order, particle i and particle $i + 1$ are spatially close and traverse similar tree branches. Tree nodes accessed by particle i remain in cache for particle $i + 1$, reducing L1/L2 cache misses.
2. **Natural parallel partitioning:** The sorted array can be split into contiguous chunks, enabling parallel subtree construction with minimal inter-task dependencies.

3.3 Leapfrog Integration (KDK Scheme)

We integrate the equations of motion using the Kick–Drift–Kick (KDK) variant of the leapfrog integrator [5, 6]. Given positions $\mathbf{x}(t)$, velocities $\mathbf{v}(t)$, and accelerations $\mathbf{a}(t)$ at time t , one timestep of size Δt proceeds as:

$$\mathbf{v}(t + \Delta t/2) = \mathbf{v}(t) + \frac{\Delta t}{2} \mathbf{a}(t) \quad (\text{Kick}) \quad (5)$$

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t \mathbf{v}(t + \Delta t/2) \quad (\text{Drift}) \quad (6)$$

$$\mathbf{a}(t + \Delta t) = F(\mathbf{x}(t + \Delta t))/m \quad (\text{Recompute forces}) \quad (7)$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t + \Delta t/2) + \frac{\Delta t}{2} \mathbf{a}(t + \Delta t) \quad (\text{Kick}) \quad (8)$$

We chose leapfrog over higher-order methods (e.g., Runge-Kutta 4) for two reasons. First, leapfrog is *symplectic*, i.e., it keeps the total energy of a system consistent over long time scales. While RK4 may be more accurate over a very short time, its energy errors accumulate, causing the system’s energy to drift. Second, Leapfrog requires only one function evaluation (acceleration calculation) per time step. Standard RK4 requires four function evaluations per step, making Leapfrog roughly four times faster for the same number of time steps. We use $\Delta t = 0.005$ in reduced units.

Once the drift is done, we reconstruct the quadtree at each step since the new positions of the particles impacts the force computation.

3.4 Initial Conditions: Plummer Model

We initialize particles using the Plummer model [8, 9], a standard astrophysical test case describing a self-gravitating star cluster in approximate virial equilibrium with density profile:

$$\rho(r) \propto \left(1 + \frac{r^2}{a^2}\right)^{-5/2} \quad (9)$$

where a is the Plummer radius (scale length). The Plummer model is generally preferred over a uniform (homogeneous) sphere model in N-body simulations primarily because it offers a more realistic, physically motivated density profile that approximates stellar systems (like globular clusters) and galaxies, whereas a uniform sphere is non-physical.

Nonetheless, we also implement a uniform random disk distribution for comparison, in which particles are placed uniformly within a circular region with small random velocities. This distribution is not in equilibrium and provides a contrasting workload profile for parallelization analysis.

3.5 OpenMP Parallelization Strategy

Each timestep involves seven phases. Table 1 summarizes the parallelization approach for each phase (if at all), the OpenMP mechanism used, and the rationale for the choice.

Table 1: Parallelization strategy per simulation phase.

Phase	OpenMP Mechanism	Schedule	Rationale
Kick (half)	<code>parallel for</code>	static	Independent per particle, uniform work
Drift	<code>parallel for</code>	static	Independent per particle, uniform work
Bounding box	<code>sequential</code>	N/A	Min/max of all positions is trivial
Morton sort	<code>sequential</code>	N/A	Key compute and sorting is trivial
Tree build	<code>omp task + atomic pool</code>	<code>taskwait</code>	Disjoint spatial partitions
Force compute	<code>parallel for</code>	runtime	Dominant phase, load-imbalanced
Kick (half)	<code>parallel for</code>	static	Independent per particle, uniform work

3.5.1 Force Computation (Primary Target)

Force computation is the primary parallelization target. Each particle’s tree traversal is independent: it reads from the shared tree (which is read-only during this phase) and writes only to its own acceleration vector. There are no data races, making this embarrassingly parallel over particles:

```
1 #pragma omp parallel for schedule(dynamic, 64)
2 for (int i = 0; i < N; i++) {
3     tree.compute_force(particles[i]);
4 }
```

Listing 1: Parallel force computation.

For clustered distributions (Plummer), particles in the dense core require deeper tree traversals than those in the outskirts, creating load imbalance. Dynamic scheduling allows idle threads to steal work from the queue, mitigating this imbalance

3.5.2 Tree Construction

Tree construction presents a more challenging parallelization problem due to the sequential dependency inherent in particle insertion: inserting particle i may subdivide a cell that particle $i+1$ needs. Our approach is inspired by the domain decomposition idea of Warren and Salmon [4]:

1. At the top levels of the tree (depth < 4), we partition the Morton-sorted particle array into four quadrant groups.
2. For each non-empty quadrant, an OpenMP task is spawned to recursively process that quadrant’s particles.
3. At depth ≥ 4 (up to 256 subtrees), each task switches to sequential insertion for its particle subset.

Thread safety is ensured without locks: each task operates on a disjoint subset of particles (guaranteed by the partitioning step), and node allocation uses `std::atomic<size_t>` for lock-free pool indexing.

3.5.3 Integration and Diagnostics

The Kick and Drift steps are trivially parallel because each particle’s update depends only on its own velocity and acceleration. We use `schedule(static)` since the work per particle is uniform.

We introduce energy diagnostics (kinetic and potential energy) to ensure correctness of the simulation by showing conservation of energy after every 10^{th} step. These are parallelized with `reduction(+:ke)` and `reduction(+:pe)` respectively. The potential energy computation is $O(N^2)$ and uses `schedule(dynamic, 16)` to handle the triangular loop structure where early iterations have more work than later ones.

4 Experimental Setup

We have used the NYU CIMS crunchy machines to run our experiments. We specifically used crunchy3 which has the following hardware details

- CPU: Four AMD Opteron 6272, giving us a total of 64 logical cores
- 8 NUMA nodes

- Memory: 256GB

The Makefile included in the project zip compiles the code using C++17 and we use OpenMP’s `omp_get_wtime()` for all of our profiling. The experiment itself is run with varying parameters as explained in the next section with each experiment being run thrice and the median time being reported. Our comparison baseline will be the Barnes-Hut algorithm run sequentially

5 Results and Analysis

5.1 Strong Scaling Analysis: Parallel Tree with Plummer Distribution

We conducted strong scaling experiments on the Plummer distribution with $\theta = 0.5$, varying thread count from 1 to 64 cores across multiple problem sizes ($N = 1K, 10K, 50K, 100K$ particles). This experiment characterizes how effectively our parallel implementation utilizes additional cores for a fixed problem size.

Table 2: Total execution time (all phases combined).

Threads	N=1K (s)	N=10K (s)	N=50K (s)	N=100K (s)
1	1.064	23.775	541.649	1873.397
2	0.549	14.639	244.808	960.445
4	0.674	7.448	134.939	513.616
8	1.011	4.209	74.793	270.066
16	0.754	9.516	49.330	166.274
64	12.620	7.475	30.346	91.594

Table 3: Force computation time.

Threads	N=1K (s)	N=10K (s)	N=50K (s)	N=100K (s)
1	0.794	10.345	112.166	253.851
2	0.327	5.617	37.479	90.739
4	0.236	2.786	17.048	47.014
8	0.198	1.416	9.699	21.508
16	0.101	1.674	6.409	11.089
64	1.195	0.774	2.975	7.047

Table 4: Speedup of force computation relative to single-thread baseline ($S(p) = T_1/T_p$).

Threads	N=1K	N=10K	N=50K	N=100K
1	1.00×	1.00×	1.00×	1.00×
2	2.43×	1.84×	2.99×	2.80×
4	3.37×	3.72×	6.57×	5.40×
8	4.01×	7.31×	11.56×	11.79×
16	7.86×	6.18×	17.50×	22.89×
64	0.66×	13.37×	37.73×	36.02×

For $N=100K$, speedup increases consistently from 1 to 16 threads ($1.00\times$ to $22.89\times$), indicating good scaling, irrespective of problem sizes. However, at 64 threads, several problem sizes show scaling anomalies ($N=1K$ shows $0.66\times$ speedup, indicating slowdown). This is explained

by the overhead of task scheduling on very small problems where total work is insufficient to amortize thread overhead costs. For larger problems (N=50K, N=100K), scaling to 64 threads remains effective, demonstrating that Barnes-Hut parallelization scales well with both problem size and thread count.

Amdahl’s law predicts maximum speedup on p processors given a serial fraction f :

$$S(p) = \frac{1}{f + (1 - f)/p} \quad (10)$$

From our measurements, we can infer the serial fraction. For N=100K at 16 threads with $S(16) = 22.89$:

$$22.89 = \frac{1}{f + (1 - f)/16} \quad (11)$$

$$f + \frac{1 - f}{16} = \frac{1}{22.89} = 0.0437 \quad (12)$$

$$f + 0.0625 - 0.0625f = 0.0437 \quad (13)$$

$$0.9375f = -0.0188 \quad (14)$$

$$f \approx -0.020 \quad (15)$$

The negative value indicates that our speedup exceeds our expectations based on Amdahl’s law. This is due to cache effects (discussed more later), i.e., parallelization with proper data layout (Morton sorting) improves per-thread memory access patterns, enabling super-linear speedup on smaller thread counts. As thread count increases to 64, speedup becomes more sublinear due to NUMA contention and synchronization overhead.

5.2 Effect of Approximation Parameter θ on Force Computation

The parameter θ controls the accuracy-speed tradeoff in the Barnes-Hut approximation. Smaller θ requires deeper tree traversals for each particle (more accurate), while larger θ allows coarser approximations (faster). We studied force computation time for N=100K particles across $\theta \in \{0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$ with threads $p \in \{4, 8, 16, 64\}$.

θ	T=4 (s)	T=8 (s)	T=16 (s)	T=64 (s)
0.2	267.249	133.237	66.713	21.500
0.4	85.451	43.891	22.277	7.222
0.5	60.681	30.092	15.434	5.129
0.6	44.954	22.314	11.741	4.011
0.8	29.094	14.727	7.398	2.640
1.0	20.773	10.724	5.637	1.991

Table 5: Force computation time (seconds) for varying θ and thread count (N=100K particles)

θ	S(8)	S(16)	S(64)	Avg Speedup
0.2	2.01×	4.01×	12.43×	6.15×
0.4	1.95×	3.84×	11.83×	5.87×
0.5	2.01×	3.93×	11.81×	5.92×
0.6	2.02×	3.83×	11.21×	5.69×
0.8	1.98×	3.94×	11.02×	5.65×
1.0	1.94×	3.69×	10.43×	5.35×
Average	1.99×	3.87×	11.45×	5.77×

Table 6: Speedup of force computation relative to 4-thread baseline for each θ value

We see that all θ values exhibit nearly identical speedup. This indicates that the opening angle parameter controls the sequential work per particle (work per timestep), but does not affect parallel scaling efficiency. The parallelization is independent of accuracy parameter i.e., dynamic scheduling handles all θ equally well.

5.3 Data Layout and Parallelism

We conducted a controlled experiment comparing sequential tree construction followed by parallel force computation (baseline) against parallel tree construction with Morton key sorting followed by parallel force computation. This isolates the impact of tree construction strategy and data layout on tree building and force computation performance.

5.3.1 Experiment Design

Both configurations run the same simulation: $N = 1,000,000$ particles initialized using the Plummer model, with the parameter $\theta = 0.7$, run using 64 threads. The only difference is that

- **Sequential tree:** Particles in original order $\rightarrow$ tree built sequentially $\rightarrow$ force computed in parallel
- **Morton sorting + Parallel tree:** Particles sorted by Morton key $\rightarrow$ tree built in parallel $\rightarrow$ force computed in parallel

5.3.2 Results and Key Findings

Table 7: Timing breakdown comparing sequential tree construction vs parallel tree construction

Component	Sequential (s)	Parallel (s)	Difference (s)	Speedup
Tree build	123.26	86.71	36.55	1.42×
Force compute	136.52	37.49	99.03	3.64×
Morton sort	—	16.55	—	—
Total	259.78	124.20	135.58	2.09×

Table 8: Hardware performance counters (measured via `perf stat`).

Metric	Sequential	Parallel + Morton	Ratio	Interpretation
IPC (insn/cycle)	0.14	0.52	$3.71\times$	CPU utilization improved
LLC miss rate (%)	20.53	1.03	$20.0\times$	Cache locality greatly improved
LLC hit rate (%)	79.47	98.97	$1.24\times$	Nodes stay in L3 cache

Table 7 shows the timing breakdown. The results display how much benefit each phase receives from parallelization. Critically, the force computation speedup dramatically exceeds the tree build speedup, indicating that the benefit extends far beyond simple parallelism. To understand this mechanism, we measured hardware performance counters using `perf stat`, shown in Table 8

- **Sequential tree (baseline):** When particles are processed in arbitrary order, each particle’s tree traversal accesses different tree nodes based on its position. Particle i at, say, $(10, 10)$ accesses nodes in quadrant $(0,0)$; particle $i + 1$ which may be at $(100, 100)$ jumps to quadrant $(3,3)$. Tree nodes accessed by particle i are evicted from L1/L2 cache before particle $i + 1$ accesses them. With 1M particles distributed across 64 threads, threads frequently access remote tree nodes. The result? 20.53% LLC miss rate, $\text{IPC} = 0.14$
- **Parallel tree + Morton sort:** When particles are sorted by Morton key (Z-order curve), spatially nearby particles are adjacent in the array. During force computation, particle i at $(10, 10)$ and particle $i + 1$ at $(10.5, 10.5)$ traverse similar or identical tree paths. Tree nodes accessed by particle i remain in L1/L2 cache for particle $i + 1$. This results in 1.03% LLC miss rate ($20\times$ improvement), $\text{IPC} = 0.52$ ($3.71\times$ improvement)

The $3.64\times$ force speedup emerges from the compound effects of parallel force computation and the cache locality gained from Morton-sorted particles sharing tree nodes across threads indicating that a simple sequential $O(N \log N)$ reordering, improves force computation even more than the benefit of plain parallelism.

6 Conclusions

This project demonstrates that effective parallelization of irregular tree-based algorithms requires attention to three often-overlooked factors:

1. **Data layout matters more than algorithm parallelization.** Morton key sorting ($O(N \log N)$, 16.6 seconds) provides $3.64\times$ speedup to force computation, exceeding the $1.42\times$ benefit from parallelizing tree construction. This generalizes: for memory-bound algorithms, spatial data organization can outweigh algorithmic improvements.
2. **Approximation parameters decouple from scaling.** The opening angle θ controls total work but not parallel efficiency; speedup curves are parallel across $\theta \in \{0.2, 1.0\}$ with $\pm 3\%$ variation. Load balancing via dynamic scheduling is robust across accuracy regimes, enabling practitioners to adjust accuracy without re-tuning parallelization.
3. **Cache effects enable super-linear speedup.** Measured speedups exceed Amdahl’s law ($36.02\times$ at 64 threads vs. predicted $20.8\times$) due to improved cache locality. Effective serial fraction becomes negative at high thread counts, validating the principle that data layout is a first-order performance determinant.

These insights generalize beyond Barnes-Hut to other tree-based algorithms (FMM, KD-trees, quadrees in spatial indexing) and irregular algorithms broadly. The methodology—combining strong scaling measurements, hardware counter analysis, and Amdahl’s law inference—provides a template for analyzing parallel algorithms beyond naive speedup metrics.

References

- [1] J. Barnes and P. Hut, “A hierarchical $O(N \log N)$ force-calculation algorithm,” *Nature*, vol. 324, pp. 446–449, 1986.
- [2] G. M. Morton, “A computer oriented geodetic data base and a new technique in file sequencing,” IBM Ottawa Technical Report, 1966.
- [3] L. Greengard and V. Rokhlin, “A fast algorithm for particle simulations,” *Journal of Computational Physics*, vol. 73, pp. 325–348, 1987.
- [4] M. S. Warren and J. K. Salmon, “A parallel hashed oct-tree N-body algorithm,” in *Proceedings of Supercomputing '93*, pp. 12–21, 1993.
- [5] L. Verlet, “Computer ‘experiments’ on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules,” *Physical Review*, vol. 159, pp. 98–103, 1967.
- [6] P. Hut, J. Makino, and S. McMillan, “Building a better leapfrog,” *Astrophysical Journal Letters*, vol. 443, pp. L93–L96, 1995.
- [7] S. J. Aarseth, “Dynamical evolution of clusters of galaxies, I,” *Monthly Notices of the Royal Astronomical Society*, vol. 126, pp. 223–255, 1963.
- [8] S. J. Aarseth, M. Hénon, and R. Wielen, “A comparison of numerical methods for the study of star field dynamics,” *Astronomy and Astrophysics*, vol. 37, pp. 183–187, 1974.
- [9] H. C. Plummer, “On the problem of distribution in globular star clusters,” *Monthly Notices of the Royal Astronomical Society*, vol. 71, pp. 460–470, 1911.